

Questão 7 — Design Patterns

Normalização de serviços de terceiros — e-mail e SMS

Objetivo

Descrever quais padrões de projeto seriam utilizados para tornar uniformes as interfaces de diferentes fornecedores de serviços externos, conforme solicitado no desafio.

1. Adapter — padrão principal

O padrão **Adapter** seria utilizado como principal solução para normalizar as diferentes interfaces disponibilizadas pelos fornecedores.

Cada fornecedor pode possuir sua própria API, nomenclatura e formato de requisição. Em vez de permitir que a regra de negócio conheça essas diferenças, seria definida uma interface interna comum, por exemplo **NotificationProvider**. Cada fornecedor teria então um adapter responsável por traduzir essa interface para o contrato específico da respectiva API externa.

Conceito:

Aplicação → NotificationProvider → Adapter → API do fornecedor

Dessa forma, o restante da aplicação depende da interface interna e não da interface específica de cada fornecedor. Isso reduz o acoplamento e isola as particularidades das integrações externas.

2. Continuidade com a Questão 1

Essa escolha também mantém uma coerência com a arquitetura apresentada na **Questão 1**. Naquela solução, o padrão **Adapter** já era utilizado para encapsular as diferenças entre os provedores externos, enquanto uma **Factory** era utilizada para centralizar a seleção/criação do provider adequado.

A mesma ideia pode ser aplicada aos serviços de e-mail e SMS: os adapters isolam os contratos específicos dos fornecedores e a Factory pode centralizar a criação ou seleção do adapter necessário.

3. Strategy — quando houver necessidade de alternância

O padrão **Strategy** poderia complementar a solução quando fosse necessário alternar entre diferentes implementações de envio. Por exemplo, um serviço poderia trabalhar com diferentes fornecedores de e-mail ou SMS conforme configuração, disponibilidade ou outro critério definido pela aplicação.

Nesse cenário, cada implementação encapsula uma estratégia de envio, enquanto o código cliente permanece desacoplado da implementação concreta.

4. Factory — criação e seleção dos adapters

A **Factory** pode complementar a solução centralizando a criação dos adapters. Por exemplo, uma configuração poderia determinar que o serviço de e-mail utilizará determinado fornecedor, e a Factory retornaria o adapter correspondente.

A Factory, portanto, não seria o padrão principal responsável pela normalização das interfaces. Seu papel seria encapsular a criação/seleção das implementações, enquanto o Adapter continuaria responsável pela tradução entre a interface interna e o contrato externo.

5. Exemplo conceitual

NotificationService	NotificationProvider
Regra de negócio	Interface interna uniforme
EmailAdapter / SMSAdapter	Implementações específicas
Fornecedor A / B / C	APIs externas

6. Justificativa final

A combinação proposta favorece **baixo acoplamento, substituição de fornecedores, testabilidade e manutenção**. As particularidades das APIs externas ficam concentradas nos adapters, enquanto a aplicação trabalha com contratos internos estáveis.

Resumo da escolha:

- **Adapter:** normaliza interfaces diferentes de fornecedores.
- **Strategy:** permite variar/trocar a implementação de envio quando necessário.
- **Factory:** pode centralizar a criação/seleção dos adapters.

A escolha é consistente com a solução da Questão 1, na qual Adapter e Factory já foram utilizados para isolar e selecionar provedores externos.